Computer Graphics

Visibility

List Priority Algorithms

Dr. Hardik Jain
Assistant Professor
Department of Computer Science and Engineering
IIT Jodhpur

1

List Priority Algorithms

Technique

- Maintain a list of all polygons/surfaces in the scene.
- Assign each surface a **priority** based on its depth relative to the viewpoint.
- Sort the list according to this priority to get a **visibility order**.
- Render the polygons sequentially in that order; because of the ordering, later polygons overwrite earlier ones correctly.
- Use object-space tests (depth comparisons, possible splitting) to repair the order when simple sorting is ambiguous.

Key Properties

- Objects may need splitting if no unique ordering exists.
- Needs expensive sorting/reordering every frame!!
- Ordering and splitting polygons: *Object-precision operation*.
- Overwriting farther points while scan conversion: *Image-precision operation*.
- Various algorithms differ in how they determine the sorted ordered as well as which objects get split and when the splitting occurs.

2

Depth Sort (Painter's) Algorithm


```
algorithm DepthSort(polygons plist):
    // Sort polygons by decreasing average depth (back-to-front)
    sort plist by avg_z(plist) descending  // avg_z = average z of vertices

    while plist is not empty:
        select poly_i from plist with maximum avg_z  // farthest remaining
        for each poly_j in plist ahead of poly_i (smaller avg_z):
            if not OrderOK(poly_i, poly_j):
                swap poly_i and poly_j in list
                restart tests for poly_i  // recheck from scratch
                if swap attempted twice: split intersecting polygon
        if all OrderOK(poly_i, *):  // safe to draw
            rasterize(poly_i)  // draw to framebuffer
            remove poly_i from P

function OrderOK(P, Q):  // P behind Q? (P farthest)
    // Test 1: No z-extent overlap
    if max_z(P) < min_z(Q): return true
    if max_z(Q) < min_z(P): return false  // swap needed

    // Test 2: No 2D projection overlap
    if no x-extent overlap or no y-extent overlap: return true

    // Test 3: P entirely behind Q's plane
    if all vertices of P behind Q.plane: return true

    // Test 4: Q entirely in front of P's plane
    if all vertices of Q in front of P.plane: return true

    return false  // ambiguous: needs swap/split
```


Depth Sort (Painter's) Algorithm

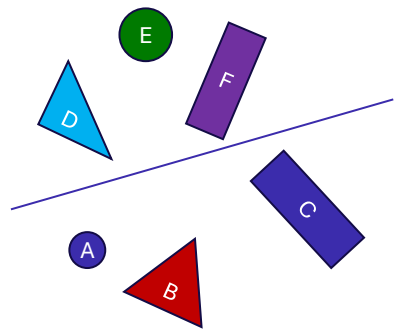
Discussion

- Ensures back-to-front ordering for proper rendering.
- No aliasing effects introduced as objects are reordered/split.
- Reordering and splitting of polygons have to be done at run time.
- Redo the whole calculations if the view-point changes.
- Computationally expensive.
- As soon as the camera is moved, the sorting has to be re-done

Discussion

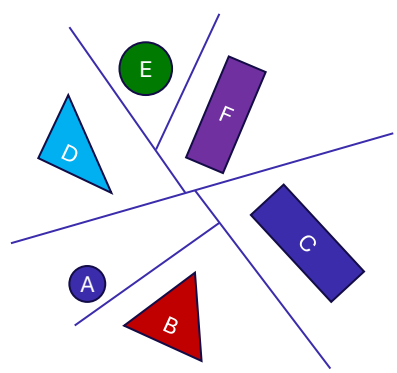
Binary Space Partitioning

- Can we have simple linear display algorithm, perhaps using expensive preprocessing?
- Consider a plane in space that divides scene into two halves.
- Objects on the same side of the plane as the eye cannot be blocked by objects on the other side.



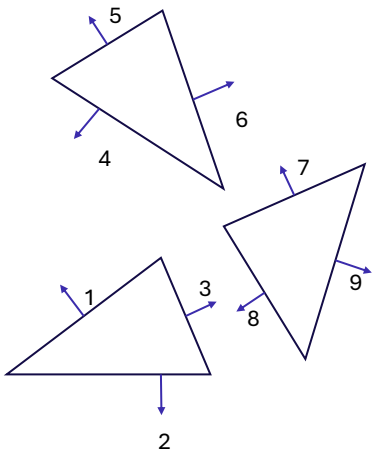
Binary Space Partitioning Trees

- Each side of the plane can further be divided using other planes till we reach a single object.
- If environments consist of clusters of objects, separate them using an appropriate plane.
- We end up with the BSP Tree representation of the scene.
- Internal nodes contain partitioning planes; leaf nodes are polygons.
- Some preprocessing to construct the tree, but simple algorithm to render using it from an viewpoint.



Binary Space Partitioning Trees

- Planes of polygons in the scene can be used as partitioning planes
- Normal direction indicates the "front" side of the plane.
- Each plane divides space into two sides.
- If a polygon lies on both sides of the plane, divide it into two parts.
- Continue this recursively till each side contains exactly one polygon.
- Choose the plane with least splitting as the root node.



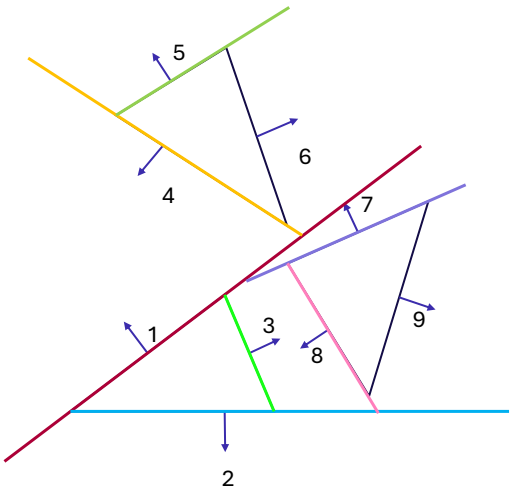
7

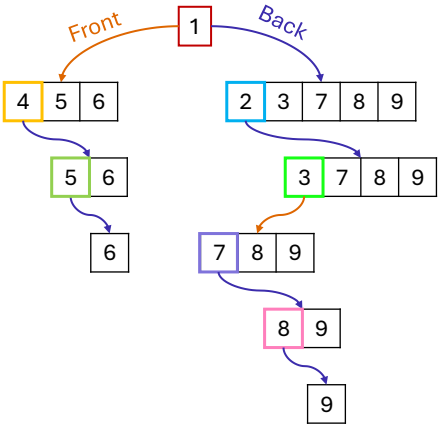
BSP Tree Creation: Pseudocode

```
struct BSPNode {
    Polygon* splitter;      // splitting plane
    BSPNode* front;         // polys with + side (normal-facing)
    BSPNode* back;          // polys with - side (normal-opposite)
}

BSPNode* makeBSPTree(List<Polygon*> plist) {
    if (plist.empty()) return NULL;  // base case

    // pick first poly as splitter
    Polygon* splitter = plist[0];    // simplistic choice
    List<Polygon*> front = empty, back = empty;

    for each poly in plist:
        if (poly == splitter) continue;
        Side s = classify(poly, splitter.plane);
        if (s == FRONT)        front.add(poly);
        else if (s == BACK)    back.add(poly);
        else {  // STRADDLING
            auto [f_frag, b_frag] = split(poly, splitter.plane);
            front.add(f_frag); back.add(b_frag);
        }

    BSPNode* root = new BSPNode(splitter);
    root->front = makeBSPTree(front);
    root->back  = makeBSPTree(back);
    return root;
}
```


8

BSP Tree Creation: Example





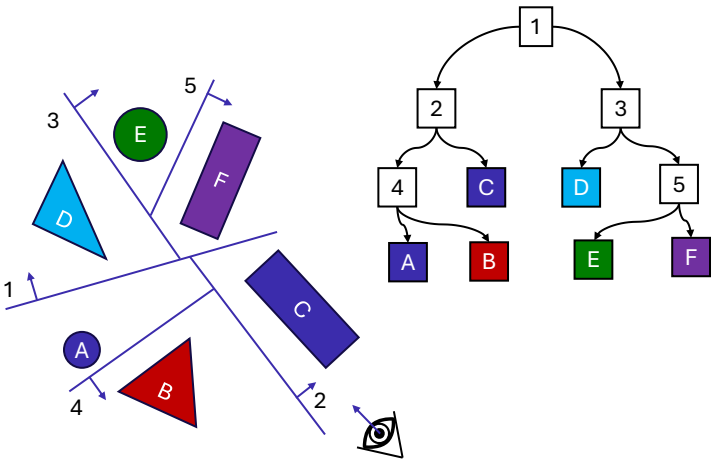
9

BSP Tree Traversal: Pseudocode

```
void PainterTraverseAndRender(BSPNode* node, Point viewer) {
    if (node == null) return;
    bool viewerFront = ViewerInFront(viewer, node->splitter.plane);

    if (viewerFront) {
       PainterTraverseAndRender(node->back,  viewer);  // hidden first
       RenderPolygon(node->splitter);        // self next
       PainterTraverseAndRender(node->front, viewer);  // visible last
    } else {
       PainterTraverseAndRender(node->front, viewer);  // hidden first
       if (no back-face culling) {
        RenderPolygon(node->splitter);       // self next
    }
    PainterTraverseAndRender(node->back,  viewer);  // visible last
    }
}

```


10

BSP Tree Traversal: Example



Current Node	viewerFront	Next Node	Render
1	F	Front	-
3	T	Back	-
D	NA	Root	D
3	Already Checked	Front	-
5	F	Back	-
E	NA	Root	E
5	Already Checked	Front	-
F	NA	Root	F
5	Both Sides Checked	Root	-
3	Both Sides Checked	Root	-
1	Front Already Checked	Back	-
2	T	Back	-
4	T	Back	-
A	NA	Root	A
4	Back Already Checked	Front	-
B	NA	Root	B
4	Both Sides Checked	Root	-
2	Back Already Checked	Front	-
C	NA	-	C

Performance Considerations

- Construction of the BSP tree is expensive.
- No splitting while rendering; everything is done during preprocessing.
- Straightforward display algorithm. Back-face culling woven into it.
- Strategy of selecting the root has a great impact.
- Select the plane that splits least number of polygons.

Visible Surface Determination

- Sorting in the right order is key to all of them.
- For quick rendering, z-buffer algorithms are better.
- BSP Trees can be fast if environment is static.
- Ease of implementation and scope of hardware acceleration are also important.
- Z-buffering is popular due to memory being cheap.

Thank You